

SERVICE ACTUEL · DATA-DEV

Développement et versionnement des flows

23 capacités analysées

services/data-dev · Kotlin 2.4.10, Spring Boot 4.1.1 et JVM 21 observés dans Gradle. Domaines CQRS, JPA/PostgreSQL, Flyway, Kafka/Protobuf, JGraphT, génération Python/SQL/PySpark, S3 et REST Gitea/IAM/Jupyter.

Service des développeurs et administrateurs de projet. Il conserve les graphes, branches et versions de définition, produit les scripts exécutables, prépare les accès aux datasets et relie projets, Git, notebooks, modules et notifications. Il conserve aussi des fonctions de projets désormais en transition vers IAM.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Flows, IAM, Datasets, Connectors, Writer, Publisher, Query, Runtimes, Applications, Notifications, Audit, Migration, Maintenance, Frontends.

Périmètre de lecture

- Contrôleurs project, definition, branch, version, internalmodule, dataset, favorite et schedule.
- Handlers des créations et mises à jour de projets, offboarding, graphes, sauvegarde, publication, preview, branches, MR et résolution.
- ScriptGeneratorServiceImpl, calcul des schémas, conversion Protobuf des versions et adaptateur DataflowForLibServiceImpl.
- Agrégats Version, DataflowDefinition, DataflowBranch, variables, modules et miroirs datasets.
- Adaptateur Gitea, intégration notebooks, visibilité IAM, chiffrement SSH, outbox, DLT, run modes et manifestes Gradle/Helm.

Objets et cycle de vie

- Project, ProjectMember, User, Group et Subscription sont des objets legacy ou miroirs IAM. Leur correspondance canonique doit précéder l'import des ressources du projet.
- Application représente Gitea/Jupyter et leur provisioning. Les effets distants et leurs erreurs ont un cycle distinct de l'enregistrement projet.
- DataflowDefinition conserve identité, projet, propriétaire et type PYTHON/PYSPARK/SQL. DataflowBranch conserve filiation, propriétaire, statut DRAFT/PRODUCTION, cluster, suppression et dernières versions.
- GraphVersion contient le graphe et ses scripts. VERSION_SAVE, VERSION_MERGE, VERSION_PUBLISH, VERSION_MODULE et VERSION_ENV_VAR décrivent des changements de définition ; CollectVersion, DeleteVersion, EmptyDatasetVersion et ColumnOperationVersion décrivent des opérations techniques.
- Script conserve dépendances, transformations/étapes, sorties, type et manifeste dataset/branche/mode. L'artefact de code S3 est une seconde ressource à sauvegarder avec sa référence.
- InternalModule possède draft et publication séparés. EnvVar possède portée globale/locale et un masquage de réponse pour secrets, sans chiffrement applicatif visible dans son DAO.
- MergeRequest conserve auteur, reviewer, branches, historique de statut et choix de suppression de source. MergeRequestResolution conserve le hash de script cible de la décision.
- DatasetBranch, Execution, Cluster et ExecutionSchedule sont des projections d'autres services. Leur stockage local ne leur donne pas l'autorité sur les données publiées.

- OutboxEvent et DeadLetterEvent portent les commandes/événements non délivrés, les tentatives et les reçus. Ils font partie du périmètre de sauvegarde et de bascule.

Dépendances du service

- IAM fournit canonicalisation des projets, décisions de visibilité et d'autorisations spécialisées, comptes de service et événements d'identité. IAM est le writer des tuples, data-dev émet des faits.
- APISIX autorise les routes humaines puis le service authentifie. Les contrôles de cohérence de payload, de propriété d'objet et certains rôles métier résiduels restent dans les handlers lus.
- Process-data reçoit VersionEvent et les triggers ; il renvoie exécutions, schedules et clusters par Kafka/Protobuf.
- Data-model fournit datasets, branches, schémas, labels et liens projet/dataset. Ses demandes de vidage/suppression/changement de colonnes peuvent provoquer des jobs.
- Data-collect émet les collectes CREATED/UPDATED ; data-dev compile leurs transformations et les rend exécutables.
- Gitea est appelé par REST pour organisation, dépôt, compte et collaboration. Jupyter passe par ApplicationServiceJupyter et événements d'applications.
- Notification-center reçoit les abonnements et notifications de MR/flows ainsi que les changements de favoris.
- PostgreSQL/JPA/Flyway porte graphes et état de commande. S3 conserve les archives et scripts. Kafka transporte les événements de contrôle ; ce transport JVM doit aussi quitter la cible finale.

Fonctionnalités et compatibilité à préserver

DD-01 **Cible non qualifiée**

Projets, espaces personnels et compatibilité avec IAM

COMPORTEMENT OBSERVÉ

Les routes historiques listent les projets avec recherche, tri, pagination, filtre onlyMine et inclusion des espaces personnels. La visibilité du demandeur passe par une décision IAM avant restitution. Les réponses exposent la correspondance avec les identifiants IAM et des en-têtes de dépréciation. La création valide manager et slug, crée les applications initiales et émet les événements projet. Un mode de passage par IAM écrit d'abord le projet central puis le miroir local. PATCH /projects/{id} répond 410. La suppression est jouable si un tombstone prouve une suppression antérieure. Un espace SINGLE exige une règle particulière d'administration pour sa suppression et sa création est conditionnée au profil utilisateur.

UTILISATEURS ET RÔLES

Développeur, responsable de projet, administrateur d'organisation et IAM.

INTERFACES DANS LE CODE

GET/POST /projects ; GET/DELETE/PATCH /projects/{id} ; GET /projects/exists ; POST /projects/list ; GET /projects/personal-space ; GET /users/{id}/projects ; GET /kpi/projects.

DONNÉES ET ÉTATS

Project et ProjectMember en PostgreSQL, visibilité PUBLIC ou SINGLE, nom historique et identifiant IAM canonique. Initialisation des applications et enregistrement de compte de service peuvent prolonger le cycle après création.

DÉPENDANCES ET ÉCHANGES

IAM par le pont d'écriture et décisions de visibilité ; événements vers consommateurs de projets ; applications Gitea/Jupyter ; enregistrement IAM du compte de service.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust possède les projets, adhésions, identifiants et espaces personnels. Flows ne garde que la référence canonique nécessaire aux graphes. Applications prend en charge les effets de provisionnement, avec reçus persistants.

MIGRATION DE CETTE CAPACITÉ

Importer les correspondances legacy/IAM, inventorier les appelants des routes dépréciées, puis fournir une façade temporaire. Déplacer les effets applicatifs dans une procédure de provisionnement reprise sur incident avant de supprimer les tables miroir devenues inutiles.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Créer un projet avec membres, interrompre le provisionnement Gitea puis reprendre. IAM garde un seul projet, les applications finissent par converger et la façade restitue le même identifiant. Un utilisateur sans droit de lecture ne modifie ni les lignes ni le total visible d'une page.

Références de l'analyse

`monorepo · services/data-dev/app/data-dev-`

`app/src/main/kotlin/com/cleyrop/data/dev/app/project/controller/ProjectController.kt:60` Routes, canonicalisation, dépréciation et arrêt du PATCH.

`monorepo · services/data-dev/app/data-dev-`

`domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/command/handler/CreateProjectWithMembersCommandHandler.kt:48` Pont IAM avant création locale et membres.

`monorepo · services/data-dev/app/data-dev-`

`domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/command/handler/DeleteProjectCommandHandler.kt:33` Idempotence de suppression et invariant SINGLE.

`monorepo · services/data-dev/app/data-dev-`

`domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/service/ProjectVisibilityService.kt:16` Filtrage par décision centrale.

DD-02 **Cible non qualifiée****Miroirs d'identité, membres et départ d'un utilisateur****COMPORTEMENT OBSERVÉ**

Les événements utilisateurs, groupes, abonnements et membres alimentent des miroirs locaux et les effets applicatifs. Lors d'une désactivation, les flows du membre et les projets dont il était manager peuvent être réaffectés. Lors d'une suppression, l'espace personnel peut être supprimé, les adhésions ordinaires retirées et les propriétaires remplacés. L'absence de remplaçant ne crée pas de nouveau propriétaire implicite. Le nettoyage Git vérifie l'identité du compte par email avant verrouillage. Une erreur Git laisse le miroir révoqué mais le compte distant inchangé et incrémente un compteur. Les rôles historiques servent encore à certaines règles métier de provisioning et d'éligibilité, même si la décision d'accès des routes est centrale.

UTILISATEURS ET RÔLES

IAM, administrateurs et consommateurs des événements de lifecycle.

INTERFACES DANS LE CODE

`OnUserDeactivatedCommandHandler` ; `OnUserDeletedCommandHandler` ;

`ReapUserFootprintCommandHandler` ; événements de membres et d'identité. `GET /projects/{id}/members` ; `GET /users/{id}/check-owner`.

DONNÉES ET ÉTATS

User, groupes, abonnements et ProjectMember en miroir ; propriétaires de projets/flows ; états des applications ; compte Git distant verrouillable.

DÉPENDANCES ET ÉCHANGES

Flux IAM ou legacy, événements de transfert de propriétaire, Gitea et applications du projet.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust émet des événements versionnés et ordonnés par identité. Flows et Applications appliquent des changements idempotents. L'offboarding devient une opération suivie jusqu'aux reçus des effets distants.

MIGRATION DE CETTE CAPACITÉ

Reprendre chaque règle de transfert, conserver les références historiques d'auteur et ne pas assimiler désactivation à effacement. Introduire une file de réconciliation du verrouillage Git, car le chemin lu signale l'échec sans réparateur automatique.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Désactiver le propriétaire pendant une indisponibilité Gitea. Les nouveaux accès et runs sont refusés, l'historique reste attribué, le transfert décidé par IAM est appliqué une seule fois et un effet de verrouillage non terminé reste visible jusqu'à son acquittement.

Références de l'analyse

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/command/handler/OnUserDeactivatedCommandHandler.kt:35` Ordre de réaffectation des flows puis managers.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/command/handler/OnUserDeletedCommandHandler.kt:34` Espace personnel, adhésions et transfert.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/command/handler/ReapUserFootprintCommandHandler.kt:76` Vérification du compte Git et échec observé sans réconciliation.

DD-03 **Cible non qualifiée****Dépôts Gitea, comptes Git et clés SSH****COMPORTEMENT OBSERVÉ**

L'initialisation d'une application Gitea crée un état WAITING et émet une commande asynchrone. L'adaptateur sait assurer l'organisation, créer/supprimer un dépôt, migrer un dépôt, gérer les collaborateurs, créer/retrouver des comptes et verrouiller leur connexion. La résolution d'un login existant tient compte du login conservé et de l'email. Le retrait d'un collaborateur peut refaire la résolution si le serveur ne connaît plus le login stocké. Les viewers et business analysts sont exclus de certains effets de collaboration. Les clés SSH locales portent un convertisseur de chiffrement et une procédure de rechiffrement existe. Ces dépôts de projet ne sont pas les branches de graphes métier, conservées en base.

UTILISATEURS ET RÔLES

Membres techniques du projet et opérateurs de migration Git.

INTERFACES DANS LE CODE

InitGitCommandHandler ; CreateGitRepositoryCommandHandler ; EditGitMemberCommandHandler ; GitCrudService ; GiteaCrudServiceImpl. Pas de préfixe public Git inventé.

DONNÉES ET ÉTATS

Application GITEA avec statut de provisioning, login Git dans le miroir utilisateur, dépôt/collaborateurs distants et SSHKey chiffrée en PostgreSQL.

DÉPENDANCES ET ÉCHANGES

REST Gitea, événements d'applications, utilisateurs/membres et chiffrement partagé avec l'infrastructure existante.

COMPORTEMENT CIBLE PROPOSÉ

Applications Rust encapsule Gitea, tiers Go compatible avec la cible zéro JVM. IAM possède l'identité. Flows référence un commit de code ou un artefact immuable quand un run dépend d'un dépôt.

MIGRATION DE CETTE CAPACITÉ

Séparer l'adaptateur Git du compilateur de graphes. Migrer les clés par une procédure contrôlée sans valeurs dans les exports. Conserver les identifiants de comptes et reçus de provisionnement pour éviter les créations concurrentes.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Rejouer deux fois la création d'application puis renommer un compte distant. Un seul dépôt et un seul compte sont liés au projet ; la suppression d'adhésion retire le bon collaborateur. Une identité Git non prouvée produit un échec explicite et aucun verrouillage d'un tiers.

Références de l'analyse

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/command/handler/InitGitCommandHandler.kt:31` Application WAITING et événement de création.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/command/handler/EditGitMemberCommandHandler.kt:32` Éligibilité, provisioning et retrait avec nouvelle résolution.

`monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/project/service/gitea/GiteaCrudServiceImpl.kt:45` Adaptateur REST organisation, dépôts et comptes.

`monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/project/repository/user/SSHKeyDAO.kt:19` Convertisseur de chiffrement de la clé.

DD-04 Cible non qualifiée

Provisionnement des notebooks et contexte de projet

COMPORTEMENT OBSERVÉ

NotebookUseCase initialise une application JUPYTER, soit immédiatement soit par événement. Avant de provisionner, il vérifie que l'application et le projet existent encore pour éviter de recréer un projet supprimé pendant la file d'attente. Il initialise ensuite manager et membres. Les erreurs métier de reprise donnent RETRY, les autres ERROR. La route de lien calcule l'URL par utilisateur et slug du projet. La route d'informations destinée au notebook vérifie la permission centrale de création, la visibilité du projet et retourne le contexte utilisateur/projet/Git. Le paramètre applicationName de cette dernière route n'est pas le mécanisme de sélection du projet dans le handler lu.

UTILISATEURS ET RÔLES

Utilisateur ouvrant un notebook, IAM et service d'applications.

INTERFACES DANS LE CODE

GET /projects/{projectId}/notebooks/link ; GET /users/me/application/{applicationName}/{projectSlug} ; NotebookUseCase.createNotebook.

DONNÉES ET ÉTATS

Application JUPYTER et statuts WAITING, RETRY ou ERROR ; associations aux membres ; contexte référencé depuis PostgreSQL.

DÉPENDANCES ET ÉCHANGES

ApplicationServiceJupyter, événements d'applications, décisions IAM et dépôt Gitea du projet.

COMPORTEMENT CIBLE PROPOSÉ

Applications et Runtimes Rust gèrent le lancement et les quotas ; l'interface notebook peut rester Python/TypeScript. Chaque accès aux données passe par les contrats Query/Writer associés au projet et au run, sans session Spark finale.

MIGRATION DE CETTE CAPACITÉ

Préserver les liens, volumes et attribution des notebooks. Remplacer le contexte de credentials ambiants par une délégation courte et bornée. Retirer les chemins Spark Connect seulement après qualification du SDK utilisateur de remplacement.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Supprimer un projet entre l'émission et le traitement du provisioning. Aucun notebook ne réapparaît. Ouvrir un notebook autorisé fournit le bon contexte canonique et les accès au stockage expirent après révocation du run.

Références de l'analyse

[monorepo](#) · [services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/usecase/NotebookUseCase.kt:101](#) Provisionnement vérifié puis RETRY/ERROR.

[monorepo](#) · [services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/project/query/handler/GetApplicationInfoQueryHandler.kt:24](#) Visibilité du projet et contexte Git.

[monorepo](#) · [services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/project/controller/UserController.kt:77](#) Décision IAM de création de notebook.

[monorepo](#) · [services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/project/controller/ProjectNotebookController.kt:33](#) Route du lien personnalisé.

DD-05 **Cible non qualifiée****Découverte des flows, versions, KPI et favoris****COMPORTEMENT OBSERVÉ**

Les contrôleurs offrent listes de flows par projet et cluster, détail avec branche/version, métadonnées, sorties, état avant publication, historique des versions et dernier script exécuté d'une transformation. Les versions ont des actions visibles ou cachées, notamment collecte, suppression, preview et retouche de colonnes. Les KPI de projets distinguent SINGLE et PUBLIC. Le favori est une association propre à l'utilisateur, idempotente à l'ajout, et émet un événement de création/suppression. L'ajout exige la visibilité de la cible si le point d'entrée n'a pas déjà imposé l'autorisation de gateway.

UTILISATEURS ET RÔLES

Développeur, frontends et administrateur.

INTERFACES DANS LE CODE

GET /dataflows ; GET /dataflows/list ; GET /dataflows/clusters/{clusterId} ; GET /dataflows/{id} ; GET /dataflows/{id}/metadata ; GET /dataflows/{id}/outputs ; GET /dataflows/{id}/branches/{branchName}/versions ; GET /kpi/dataflows ; POST /users/favorite.

DONNÉES ET ÉTATS

Définitions, branches, versions, miroir d'exécutions et favoris en PostgreSQL. Les actions techniques ont un indicateur unshown ; elles ne doivent pas être perdues à cause du filtrage d'interface.

DÉPENDANCES ET ÉCHANGES

Miroirs process-data pour l'exécution ; Notifications pour les favoris et abonnements ; IAM/gateway pour la visibilité.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust expose les contrats de recherche et l'historique des définitions. Audit conserve les opérations techniques. Frontends garde les vues distinctes entre versions de définition, runs et éditions de données approuvées.

MIGRATION DE CETTE CAPACITÉ

Construire une matrice de champs, ordres, pagination et filtrage pour chaque route. Importer l'historique caché avec son type d'origine. Migrer les favoris sans les convertir en droits persistants.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Comparer les pages et filtres sur un jeu comportant plusieurs branches, versions cachées et projets non visibles. Les totaux restent cohérents. Un favori ajouté deux fois ne crée qu'une relation et ne redonne pas accès après révocation.

Références de l'analyse

`monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/controller/DataflowGetterController.kt:77` Parcours de lecture et sorties.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/version/aggregate/Version.kt:19` Actions de version et visibilité.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/favorite/command/UserCreateFavoriteCommandHandler.kt:37` Idempotence et décision de visibilité des favoris.

`monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/version/controller/VersionController.kt:26` Historique par branche.

DD-06 **Cible non qualifiée****Sauvegarde de graphes et concurrence d'édition****COMPORTEMENT OBSERVÉ**

POST /dataflows crée ou sauvegarde un flow. À la création, owner doit désigner l'appelant. À la sauvegarde, le handler charge le flow et refuse de changer son projet ou son propriétaire par ce parcours. Le type et le cluster doivent être compatibles. Le graphe est validé avant génération. Un baseVersionId optionnel provoque un verrou de sauvegarde puis une comparaison avec la dernière version. Un décalage lève un conflit. Le diff distingue changement fonctionnel, métadonnées et absence de changement. Sans baseVersionId, certains changements de métadonnées actualisent la version existante ; avec cette précondition une nouvelle version est générée. Un changement fonctionnel de main déjà en production doit passer par proposition.

UTILISATEURS ET RÔLES

Auteur et collaborateurs du flow.

INTERFACES DANS LE CODE

POST /dataflows ; DataflowCreateCommandHandler ; DataflowSaveConflictAdvice.

DONNÉES ET ÉTATS

DataflowDefinition, DataflowBranch et GraphVersion. Le graphe contient entrées, transformations, sorties, arêtes et positions. Le graphe sauvegardé et ses scripts sont liés à une version, mais le chemin historique peut actualiser une version pour un changement de métadonnées.

DÉPENDANCES ET ÉCHANGES

Cluster miroir, variables, compilateur de scripts, S3, événements de versions et faits d'autorisation.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust sépare draft mutable et révision exécutable immuable. Chaque sauvegarde porte une révision attendue. Le manifeste compilé fixe code, schémas, dépendances, paramètres non secrets et références d'entrées.

MIGRATION DE CETTE CAPACITÉ

Importer les graphes et actions de version sans réécrire les anciennes preuves. Rendre la précondition obligatoire dans le nouveau contrat. Conserver un adaptateur de sauvegarde legacy pendant la migration de l'éditeur TypeScript.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux collaborateurs sauvegardent le même draft à partir de la même révision. Une seule écriture gagne et l'autre reçoit un conflit avec la révision courante. Le manifeste d'un run déjà lancé reste identique après déplacement de nœuds ou changement de cluster.

Références de l'analyse

`monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/controller/DataflowDevelopController.kt:107` Transaction, auteur authentifié et canonicalisation du projet.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowCreateCommandHandler.kt:153` Refus des changements de propriétaire/projet.

`monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowCreateCommandHandler.kt:456` Verrou, précondition, diff et règles de production.

DD-07 **Cible non qualifiée****Règles du graphe et compilation des transformations****COMPORTEMENT OBSERVÉ**

GraphValidator refuse les imports wildcard. PYSPARK et SQL exigent exactement une sortie par transformation ; une sortie doit avoir une transformation productrice. Python accepte plusieurs sorties. Le compilateur calcule les dépendances par les entrées, sorties et transformations amont, parcourt les transformations prêtes et propage leurs schémas. Il rend des templates différents pour Python, PySpark et SQL. SQL reste aujourd'hui une exécution PySpark dans le contrat Protobuf. Chaque transformation reçoit un ScriptId, un type, des dépendances, des sorties et éventuellement un cluster. Les archives contiennent main.py et les modules publiés du projet, puis sont écrites en S3 sous la version et le script.

UTILISATEURS ET RÔLES

Développeur de flow, éditeur et exécutateur.

INTERFACES DANS LE CODE

ScriptGeneratorService.generateGraphVersion ; GraphValidator.valid ; GraphExecutionEventProducer ; GET /dataflows/{id}/transformations/{tid} pour le contrat pylib.

DONNÉES ET ÉTATS

GraphVersion avec scripts ; artefacts S3 par version/script ; dépendances de scripts ; mappings des transformations et étapes.

DÉPENDANCES ET ÉCHANGES

JGraphT, templates Python/SQL, modules internes, schémas des datasets, S3 et process-data via VersionEvent.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust compile un graphe typé indépendant du moteur. Les opérateurs tabulaires produisent un plan SQL/Arrow exécuté par Query/Runtimes Rust. Python utilisateur reste possible dans un processus isolé. Aucune dépendance finale à PySpark, Spark Connect ou runtime JVM.

MIGRATION DE CETTE CAPACITÉ

Classer chaque flow par langage et fonctions utilisées. Transpiler les opérateurs déclaratifs et traiter les appels Spark spécifiques comme incompatibilités explicites. Conserver les artefacts historiques ; produire de nouveaux artefacts avec empreinte et version du compilateur.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Pour un graphe Python à deux sorties et un graphe SQL à jointure, vérifier ordre de dépendances, schémas, résultats et erreurs équivalentes. Une fonction exclusivement Spark est signalée avant activation du runtime Rust, avec localisation de la transformation.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/aggregate/graph/GraphValidator.kt:13](#) Contraintes wildcard et nombre de sorties.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorServiceImpl.kt:229](#) Calcul des dépendances et Script par transformation.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorServiceImpl.kt:420](#) Parcours et propagation de schémas.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorServiceImpl.kt:776](#) Archive et modules publiés.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/event/producer/GraphExecutionEventProducer.kt:107](#) SQL encodé comme PYSPARK.

DD-08 **Cible non qualifiée**

Transformations low-code et schémas intermédiaires**COMPORTEMENT OBSERVÉ**

Le vocabulaire low-code comprend filtrage/renommage/standardisation de colonnes, group by, jointures gauche/droite/interne/externe, colonne conditionnelle, suppression de doublons, filtres de lignes, concaténation, calcul mathématique, cast, union, calcul et recherche textuelle, top N et preview. Un alias historique FILTER_COLUMN reste accepté. Les étapes sont ordonnées, valident leur payload et calculent leurs schémas avant génération. Les opérations à deux entrées doivent nommer des sources disponibles. Les endpoints generate et validate opèrent sur un graphe et une étape cible ; si un flow existant est nommé, son projet doit correspondre au projet du payload.

UTILISATEURS ET RÔLES

Utilisateur de l'éditeur visuel et concepteur de modèles de transformation.

INTERFACES DANS LE CODE

POST /dataflows/generate ; POST /dataflows/validate ; StepProcessorServiceFactoryImpl et DataflowGraphProcessorServiceImpl.

DONNÉES ET ÉTATS

Étapes typées avec identifiant, ordre et payload ; OwnerSchemasCompute avec schéma rattaché au nœud/étape ; pas d'édition Iceberg créée par la seule validation.

DÉPENDANCES ET ÉCHANGES

Schémas des branches datasets, factory des processors et templates PySpark.

COMPORTEMENT CIBLE PROPOSÉ

Flows et Query Rust portent un registre versionné d'opérateurs. Chaque opérateur définit types, nulls, dates, ordre, validation et lowering vers le moteur sans JVM. L'interface TypeScript consomme le même contrat de schéma.

MIGRATION DE CETTE CAPACITÉ

Établir une fiche de parité par opérateur, y compris null, timezone, doublons, casts invalides, ordre des colonnes et stabilité top N. Garder les identifiants d'étapes et alias de payload pendant l'import.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Exécuter une campagne par opérateur sur types mixtes et valeurs limites. generate et validate du même draft annoncent le schéma réellement obtenu à l'exécution. Une jointure désignant une entrée absente échoue avant création d'un job.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/aggregate/graph/step/LowCodeTypeEnum.kt:5](#) Inventaire exhaustif des opérateurs et alias.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorService.kt:101](#) Validation des entrées et calcul des schémas avant génération.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/factory/DataflowGraphProcessorService.kt:116](#) Parcours récursif jusqu'à l'étape cible.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/controller/DataflowSchemaCompute.kt:70](#) Cohérence projet/flow du payload.

DD-09 **Cible non qualifiée**

Manifestes d'accès et séparation des branches de données**COMPORTEMENT OBSERVÉ**

Chaque script généré porte une liste d'entrées dataset/branche et de sorties dataset/branche/mode. APPEND et OVERRIDE deviennent APPEND et REPLACE. Les versions techniques de suppression utilisent DELETE. Le manifeste est sérialisé dans VersionEvent et conservé côté exécution. DataflowForLibService calcule les entrées/sorties de la transformation et choisit aujourd'hui main pour une branche en production ou une branche Nessie dérivée de l'identifiant de branche pour un draft. Sur une sortie en production, le schemaMode est relu depuis le dataset main. Les opérations techniques peuvent porter executionProjectId pour ne pas traiter leur référence synthétique de dataset comme un vrai projet.

UTILISATEURS ET RÔLES

Compilateur, service d'exécution, bibliothèque Python et services d'autorisation du stockage.

INTERFACES DANS LE CODE

ExecutionStorageManifest ; DatasetInputAccess ; DatasetOutputAccess ; VersionEvent.storageManifest ; DataflowForLibServiceImpl.getTransformationFromBranch.

DONNÉES ET ÉTATS

Manifeste par script et metadata execution_project_id pour versions techniques. Les branches métier, les branches de datasets et les références de catalogue doivent rester distinguées.

DÉPENDANCES ET ÉCHANGES

Miroir de schémas data-model, process-data, S3 et catalogue actuel Nessie dans le runtime.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust produit un manifeste immuable qui fixe les éditions d'entrée autorisées et les huit kinds cible. Writer de confiance autorise le staging exact ; Publisher décide la publication Iceberg. Le worker reçoit uniquement la portée nécessaire, sans table.modify générique qui permettrait aussi drop/commit.

MIGRATION DE CETTE CAPACITÉ

Reprendre les manifestes déjà présents et mettre à part les versions anciennes nulles. Un backfill s'appuie sur la version historique prouvée, jamais sur le draft courant. Adapter les références Nessie vers UUID/ref/schema/spec Iceberg et garder l'identité d'origine dans origin.* du candidat.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un script qui tente une écriture vers un dataset hors manifeste est refusé. Une ancienne version sans preuve de ses entrées ne reçoit pas d'accès large. Un candidat conserve les références d'édition de ses entrées après une publication concurrente de main.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorServiceImpl.kt:309](#) Construction du manifeste par transformation.

[monorepo · services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/DataflowForLibServiceImpl.kt:55](#) Branches Nessie et schemaMode production.

[monorepo · services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/definition/event/producer/GraphExecutionEventProducer.kt:144](#) Manifeste dans le contrat Protobuf.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowCreateOrUpdateCollectDatasetCommandHandler.kt:45](#) Projet réel des versions techniques.

DD-10 **Cible non qualifiée****Variables globales, locales et valeurs secrètes****COMPORTEMENT OBSERVÉ**

Le flow porte des variables globales et chaque branche des variables locales. Les globales ne changent que depuis main. Leur modification déclenche une génération VERSION_ENV_VAR sur les autres branches. Un secret reçu sans nouvelle valeur conserve la valeur stockée par son identifiant, alors qu'une valeur absente sans remplacement possible est refusée. Les doublons de clés globales sont vérifiés. Le DTO masque un secret par une chaîne vide. Le DAO stocke toutefois la valeur dans une colonne String sans convertisseur de chiffrement visible, et le générateur transforme les variables en code d'environnement inclus dans le script. Le masquage d'API ne prouve donc pas un chiffrement ni une absence du secret dans les artefacts.

UTILISATEURS ET RÔLES

Auteur du flow et opérateur des secrets.

INTERFACES DANS LE CODE

EnvVarDTO dans POST /dataflows ; EnvVarCommand.toEnvVar ;
DataflowCreateCommandHandler.buildEnvVars.

DONNÉES ET ÉTATS

EnvVar avec portée, clé, description, isSecret et valeur en PostgreSQL ; versions régénérées et scripts S3.

DÉPENDANCES ET ÉCHANGES

Générateur de scripts et bus de versionnement automatique.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust conserve les noms et références de secrets ; Runtimes résout une version de secret au lancement via IAM. Le manifeste et les artefacts publiés contiennent la référence et une preuve de version, jamais la valeur. Les logs et erreurs n'exposent pas cette valeur.

MIGRATION DE CETTE CAPACITÉ

Inventorier les variables et artefacts qui peuvent incorporer des secrets sans exporter leur contenu. Migrer vers un magasin chiffré et faire tourner les secrets concernés selon une procédure contrôlée. Fixer la règle de priorité entre portée globale et locale dans le contrat.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une sauvegarde d'un secret masqué préserve sa référence. Un changement global crée de nouvelles révisions pour les branches concernées. Une recherche sur les artefacts et événements du nouveau runtime ne retrouve aucune valeur secrète injectée au lancement.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowCreateCommandHandler.kt:74](#) Conservation d'une valeur secrète omise.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowCreateCommandHandler.kt:251](#) Portées, doublons et régénération des autres branches.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/controller/dto/EnvVarDTO.kt:39](#) Masquage en restitution.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/repository/dao/EnvVarDAO.kt:18](#) Valeur String sans convertisseur visible.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorServiceImpl.kt:431](#) Variables rendues dans les scripts.

DD-11 **Cible non qualifiée****Branches de développement et restauration contrôlée****COMPORTEMENT OBSERVÉ**

Une branche naît d'une branche source existante, après validation du flow, du nom et de l'unicité active. Le nom doit satisfaire à la fois les contraintes Git historiques et Spark ; main est réservé au parcours prévu. La création d'une branche précédemment supprimée peut réutiliser son identifiant puis la restaurer explicitement. Le simple upsert de graphe refuse au contraire de ressusciter une branche supprimée. La branche reçoit un

propriétaire, un graphe copié puis régénéré, des versions et une configuration de cluster. La suppression clôt les MR source ou cible non finales et émet les faits d'autorisation correspondants. Le handler contient encore un contrôle local de propriétaire de branche ou de flow.

UTILISATEURS ET RÔLES

Développeur et propriétaire du flow.

INTERFACES DANS LE CODE

GET /dataflows/{id}/branches ; GET /dataflows/{id}/branches/exist ; POST /dataflows/{id}/branches ; DELETE /dataflows/{id}/branches/{branchName} ; PUT /dataflows/{id}/branches/{branchName}/cluster.

DONNÉES ET ÉTATS

DataflowBranch avec sourceld, statut, owner, deleted, lastMergedAt, cluster et versions. Noms de branche main et personnalisés.

DÉPENDANCES ET ÉCHANGES

Générateur, variables, versions, MR, cluster miroir et IAM par faits de branches.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust conserve la filiation et les droits de branches. La branche de définition devient indépendante des contraintes de nom de Spark et des refs Iceberg de staging, ces dernières étant produites par Writer/Publisher.

MIGRATION DE CETTE CAPACITÉ

Conserver les IDs, noms et tombstones. Retirer les contraintes de nom propres à Spark seulement avec un mapping explicite. Clarifier dans IAM les règles de suppression encore contrôlées localement pour éviter un changement de droit durant le transfert.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Créer, supprimer puis recréer une branche par la route dédiée préserve la filiation attendue. Un upsert ordinaire du même nom reste refusé. Les MR touchées se ferment ou se retargetent selon le contrat, sans référence orpheline.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/DataflowBranchCreateCommandHandler.kt:43](#) Validation, copie, restauration et nouvelle version.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/DataflowBranchDeleteCommandHandler.kt:36](#) Contrôle local résiduel et clôture des MR.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/aggregate/DataflowBranch.kt:134](#) Contraintes combinées Git/Spark.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowCreateCommandHandler.kt:226](#) Refus de résurrection par upsert.

DD-12 Cible non qualifiée

Propositions de fusion, diff et résolution des conflits

COMPORTEMENT OBSERVÉ

Une MR référence source, cible, auteur, reviewer, titre, description et suppression éventuelle de source. La création vérifie que le demandeur possède la source, que le reviewer existe et appartient au projet, puis calcule le statut de conflit. Elle émet des abonnements, notifications et faits d'autorisation. Les API listent les

MR, les comptent, retournent leurs changements et le contenu d'un nœud. La résolution applique un choix de nœud au graphe source et conserve le hash du script cible contre lequel la décision a été prise. Une MR finale n'est plus résoluble. L'appartenance de la MR au flow de l'URI est vérifiée. Les diffs distinguent entrées, sorties, arêtes, code et étapes low-code.

UTILISATEURS ET RÔLES

Auteur, reviewer et collaborateurs du projet.

INTERFACES DANS LE CODE

POST/GET /dataflows/{id}/merge_requests ; GET /dataflows/{id}/merge_requests/{mrId} ; GET /dataflows/{id}/merge_requests/{mrId}/content/{nodeId} ; POST /dataflows/{id}/merge_requests/{mrId}/resolve ; GET /dataflows/{id}/merge_requests/countOpen.

DONNÉES ET ÉTATS

DataflowMergeRequestDefinition, historique de statuts et MergeRequestResolution avec auteur, date et hash du script cible ; nouvelles versions de la source après résolution.

DÉPENDANCES ET ÉCHANGES

GraphDiffCalculator, ConflictResolutionApplier, versions et modules compilés, Notifications et IAM.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust reprend le diff sémantique et associe chaque revue aux révisions source/cible exactes. Audit conserve les décisions de résolution. Une revue de définition reste distincte de l'audit d'un candidat de données.

MIGRATION DE CETTE CAPACITÉ

Importer les MR ouvertes et leurs preuves de résolution. Conserver les identifiants des nœuds et les hashes déjà produits. Lors du basculement, recalculer les conflits contre les révisions importées et invalider les revues dont la cible a changé.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Résoudre un conflit, modifier ensuite la cible puis tenter la fusion. La résolution obsolète est détectée et une nouvelle revue est demandée. Une MR d'un autre flow donne la même absence que si elle n'existait pas dans le flow demandé.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/DataflowMergeRequestCreateCommandHandler.kt:71](#) Validations de création, droits métier et notifications.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/ResolveConflictsCommandHandler.kt:54](#) Application, version et hash de cible.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/aggregate/DataflowMergeRequestDefinition.kt:55](#) Appartenance au flow.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/branch/controller/MergeRequestController.kt:60](#) Routes de revue, diff et résolution.

DD-13 Cible non qualifiée

Approbation, fusion et maintenance des MR dépendantes

COMPORTEMENT OBSERVÉ

L'approbation charge le flow et la MR, vérifie leur relation, puis construit une version `VERSION_MERGE` sur la branche cible à partir du graphe fusionné et des variables de la cible. Un refus doit porter un commentaire. Après fusion, la source peut être supprimée ou conserver `lastMergedAt`. Si elle disparaît, les MR qui portaient de cette source sont clôturées ; celles qui la visaient sont retargetées vers la nouvelle cible, sauf si cela ferait de la branche sa propre source. Les triggers devenus sans objet peuvent être retirés. Le retour indique aussi si les clusters source et cible sont identiques. Un recalcul des statuts des autres MR est tenté après la fusion et son échec est journalisé.

UTILISATEURS ET RÔLES

Reviewer, auteur et responsable de production.

INTERFACES DANS LE CODE

POST `/dataflows/{id}/merge_requests/{mrid}/approval` ; `DataflowMergeRequestApproveCommandHandler`.

DONNÉES ET ÉTATS

Statut final `MERGED` ou `DELETED`, nouvelle version de cible, source supprimée ou `lastMergedAt` mis à jour, MR restantes modifiées.

DÉPENDANCES ET ÉCHANGES

`BranchService` pour fusion, variables et scripts, triggers, versions et événement exécutable.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust réalise la fusion par comparaison atomique des révisions approuvées. Les retargets et suppressions font partie d'une commande durable. Audit enregistre qui a approuvé quel diff et la révision résultante.

MIGRATION DE CETTE CAPACITÉ

Porter les règles de retarget des MR en chaîne. Faire des échecs de recalcul une tâche persistante observable. Préserver le comportement des variables de cible et clarifier le changement de profil runtime entre source et cible.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Fusionner A dans main avec suppression de A alors qu'une MR B vise A. La MR B vise ensuite main ; une MR main vers A est clôturée. Une panne après création de la révision cible permet de reprendre les retargets sans deuxième fusion.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/DataflowMergeRequestApproveCommandHandler.kt:122](#)
Version cible et variables utilisées.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/DataflowMergeRequestApproveCommandHandler.kt:176](#)
Clôture et retarget des MR dépendantes.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/DataflowMergeRequestApproveCommandHandler.kt:292](#)
Ordre de fusion, statut et recalcul.

Mise en production d'une définition de flow

COMPORTEMENT OBSERVÉ

La publication cible aujourd'hui main. Elle refuse un flow absent ou supprimé, lit la dernière exécution pour le résumé, passe la branche au statut PRODUCTION et génère VERSION_PUBLISH. Le script et les sorties sont ensuite émis vers process-data. Le résumé renseigne la dernière exécution et si elle a démarré après la dernière modification. Dans le handler lu, ces indicateurs ne constituent pas une exigence de succès avant publication. L'opération publie donc une définition exécutable, pas une édition de données ayant franchi un audit WAP. La route lazy fournit l'état préalable sans exécuter cette transition.

UTILISATEURS ET RÔLES

Propriétaire habilité à publier le flow.

INTERFACES DANS LE CODE

POST /dataflows/{id}/publish ; GET /dataflows/{id}/publish/lazy ; DataflowPublishCommandHandler.

DONNÉES ET ÉTATS

DataflowStatus DRAFT/PRODUCTION sur la branche ; GraphVersion VERSION_PUBLISH ; information d'exécution distincte.

DÉPENDANCES ET ÉCHANGES

Versions, variables, miroir des exécutions et producteur GraphExecutionEvent.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust publie une révision de définition. Publisher Rust publie séparément les résultats de chaque run après audit. Les huit kinds, y compris STREAM, suivent WAP ; la réussite du job ne détermine pas à elle seule l'autorité des données.

MIGRATION DE CETTE CAPACITÉ

Renommer les concepts d'interface et de contrat pour garder publication du flow et publication des résultats distinctes. Importer le statut de production existant comme activation de définition, sans le convertir en preuve d'approbation d'un snapshot.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Activer une définition puis exécuter un run dont l'audit échoue. La définition demeure activée, le candidat reste non publié et les lecteurs restent épinglés sur la dernière édition approuvée. L'historique distingue les deux décisions.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowPublishCommandHandler.kt:112](#) Résumé lazy et publication effective.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowPublishCommandHandler.kt:131](#) Passage PRODUCTION et VERSION_PUBLISH.

[monorepo · services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/definition/controller/DataflowDevelopController.kt:186](#) Route publique de mise en production.

Preview d'une étape low-code et preuve de portée

COMPORTEMENT OBSERVÉ

Le preview charge le flow et la branche, utilise le graphe fourni ou le dernier graphe sauvegardé, exige une transformation low-code et une étape existante. Les datasets amont doivent avoir un état SUCCESS dans les branches attendues. Le graphe est tronqué à l'étape demandée avec un nouvel identifiant, puis le compilateur produit une version VERSION_PREVIEW et l'émet. Un endpoint interne destiné à Data Serve retourne les entrées d'un preview seulement si la version persistée est de ce type et si tous ses scripts ont un manifeste. Il vérifie en plus le client du bearer. Le chemin de génération lu ne sauvegarde pas directement cette version ; ce raccord de persistance est une question ouverte vérifiée dans le code.

UTILISATEURS ET RÔLES

Utilisateur de l'éditeur, runtime de preview et Data Serve.

INTERFACES DANS LE CODE

POST /dataflows/{id}/preview ; GET /internal/dataflow-versions/{versionId}/preview-scope.

DONNÉES ET ÉTATS

Graphe tronqué, identifiant de preview, scripts et artefacts temporaires ; PreviewScope lié aux manifestes persistés attendus.

DÉPENDANCES ET ÉCHANGES

Schémas/états des datasets, générateur, process-data et contrôle Data Serve.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust crée un RunSpec de preview immuable et persistant. Runtimes applique bornes de lignes, temps et mémoire ; Writer ne publie aucune édition de production. Query n'ouvre le résultat qu'après vérification de sa portée et de sa durée de rétention.

MIGRATION DE CETTE CAPACITÉ

Corriger ou établir le raccord entre émission et stockage du preview avant reprise fonctionnelle. Faire du résultat un artefact associé au run et non une permission fondée sur un UUID fourni par le client.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Créer un preview d'une étape intermédiaire, perdre le navigateur puis reprendre par son ID. Les mêmes entrées épinglées sont restituées, le scope reste disponible au consommateur autorisé et un bearer humain direct reçoit un refus.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowPreviewCommandHandler.kt:113](#) Construction et émission du preview.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/version/query/PreviewScopeQueryHandler.kt:38](#) Exigence de version persistée et manifeste complet.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/version/controller/PreviewScopeController.kt:52](#) Client interne autorisé et 404.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/event/producer/GraphExecutionEventProducer.kt:48](#) Émission outbox sans persistance de version dans le producteur.

Modules Python réutilisables et republication des flows

COMPORTEMENT OBSERVÉ

Un module interne appartient à un projet et n'accepte que PYTHON ou PYSARK. Son nom suit des limites, une liste de mots interdits et les contraintes de nom de fichier importable. La sauvegarde distingue script/description/snippet publiés et drafts ; les imports wildcard sont refusés. La publication inscrit publicationId, publishedBy et lastPublished et copie le draft comme script publié. La suppression et la publication déclenchent une régénération asynchrone de toutes les branches du projet. Les archives de run chargent les modules publiés par type. isUsableInRun repose sur publicationId afin qu'une attribution historique manquante n'efface pas l'usage d'un module publié.

UTILISATEURS ET RÔLES

Auteur de module, publisher du projet et utilisateurs des flows.

INTERFACES DANS LE CODE

GET/POST /projects/{projectId}/internal-modules ; GET /projects/{projectId}/internal-modules/{moduleId} ; POST/DELETE /projects/{projectId}/internal-modules/{id} ; POST /projects/{projectId}/internal-modules/{id}/publish ; GET /projects/{projectId}/internal-modules/exists.

DONNÉES ET ÉTATS

InternalModule avec champs draft/publiés et identité de publication ; versions VERSION_MODULE régénérées pour les branches du projet.

DÉPENDANCES ET ÉCHANGES

Registre de modules, compilateur d'archives, événement automatique par branche, IAM par faits de module.

COMPORTEMENT CIBLE PROPOSÉ

Flows et Runtimes Rust portent un registre d'artefacts de modules immuables avec digest, dépendances et provenance. Python utilisateur reste supporté ; les modules PySpark doivent migrer vers les API de données sans JVM.

MIGRATION DE CETTE CAPACITÉ

Importer chaque version publiée séparément du draft. Capturer les dépendances exactes d'un run au lieu de recharger une liste mutable de modules. Remplacer la régénération globale par des dépendances déclarées tout en gardant une réconciliation des branches affectées.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Publier un module pendant qu'un ancien run est en cours. Ce run conserve l'ancien digest ; un nouveau run utilise le nouveau. Un module draft ou un nom de fichier invalide n'entre pas dans l'archive exécutable.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/internalmodule/command/InternalModuleCommandHandler.kt:86](#) Validation, sauvegarde de draft et publication.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/internalmodule/aggregate/InternalModule.kt:98](#) Champs de versions et règle isUsableInRun.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/internalmodule/service/AsynchPublisherImpl.kt:20](#) Régénération des branches du projet.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorServiceImpl.kt:800](#) Modules publiés inclus dans les artefacts.

DD-17 **Cible non qualifiée****Métadonnées, transfert de propriétaire et faits d'autorisation****COMPORTEMENT OBSERVÉ**

La mise à jour des métadonnées change nom et description et peut modifier le trigger. Elle refuse un owner différent du propriétaire courant. Le transfert dispose d'une route distincte. Le nouveau propriétaire doit exister, être éligible selon le rôle métier historique et appartenir au projet. Une mise à jour conditionnelle compare l'ancien owner ; zéro ligne modifiée produit une raison explicite, dont conflit concurrent. Le controller ouvre une transaction couvrant le changement et ses événements. Le writer de transfert associe l'événement consommé par process-data au fait d'autorisation consommé par IAM. Une réaffectation après départ de membre emprunte également ce mécanisme spécialisé.

UTILISATEURS ET RÔLES

Propriétaire du flow, propriétaire de projet autorisé et workflow IAM d'offboarding.

INTERFACES DANS LE CODE

POST /dataflows/{id}/metadata ; PATCH /dataflows/{dataflowId}/owner/{newOwnerId} ;
ChangeDataflowOwnerCommandHandler.

DONNÉES ET ÉTATS

ownerId courant, version d'autorité attendue au transfert, métadonnées et faits outbox.

DÉPENDANCES ET ÉCHANGES

IAM/APISIX pour la permission de route ; miroirs users/members pour règles métier ; process-data et IAM pour propagation.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust décide les droits et éligibilités. Flows Rust conserve la propriété métier et enregistre transfert plus événements dans une même transaction. L'identité capturée d'un run existant ne change pas lors d'un transfert ultérieur.

MIGRATION DE CETTE CAPACITÉ

Conserver la séparation entre auteur de révision, propriétaire courant et principal d'exécution. Supprimer les contrôles de rôle hérités seulement après définition d'une règle centrale équivalente.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un collaborateur sauvegarde un graphe sans devenir propriétaire. Deux transferts concurrents ne produisent qu'un gagnant et une seule chaîne d'autorité cohérente. Un ancien run garde l'identité responsable de son lancement.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowUpdateMetadataCommandHandler.kt:130](#)
Refus de transfert via métadonnées.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/ChangeDataflowOwnerCommandHandler.kt:51](#) Éligibilité et compare-and-set de propriétaire.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/controller/DataflowDevelopController.kt:229](#) Transaction de transfert avec rollback.

DD-18 **Cible non qualifiée**

Déclencheurs fonctionnels et miroir de planification

COMPORTEMENT OBSERVÉ

Les métadonnées d'un flow peuvent référencer un dataset, un autre flow ou un chemin de fichiers du projet comme trigger. Le handler vérifie l'existence du dataset ou du flow et refuse les flows supprimés ; il émet création, modification ou suppression du trigger selon la différence avec la valeur existante. La liste des triggers possibles parcourt toutes les pages des flows du projet, exclut le flow courant et expose aujourd'hui un choix de chemin racine. Les événements de schedules provenant de process-data alimentent le miroir de développement. L'exécution des cron n'appartient pas à ce service.

UTILISATEURS ET RÔLES

Développeur configurant une dépendance événementielle.

INTERFACES DANS LE CODE

GET /dataflows/{id}/triggers ; POST /dataflows/{id}/metadata ; TriggerAtEventDatasetUpdateCommand, TriggerAtEventDataflowUpdateCommand et TriggerAtEventProjectFileUpdateCommand.

DONNÉES ET ÉTATS

TriggerAtEvent unique associé au flow, dataset/flow déclencheur ou projet/chemin ; miroir ExecutionSchedule.

DÉPENDANCES ET ÉCHANGES

Événements de triggers vers process-data, miroirs de datasets/flows et retour des événements de schedule.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust possède la définition des dépendances et Scheduler dans Runtimes produit des commandes idempotentes. Une dépendance de données se déclenche sur une publication approuvée, avec son édition et sa clé de déduplication.

MIGRATION DE CETTE CAPACITÉ

Inventorier cycles et changements de triggers, fixer la règle d'événement attendu pour les anciennes dépendances et importer les chemins. Pour STREAM, exiger un journal durable avant gouvernance et découper des runs bornés par epoch/range.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Rejouer une notification de publication et un événement de fichier. Chaque occurrence légitime crée au plus un run ; une occurrence dupliquée n'en crée pas. Un candidat refusé par audit ne déclenche aucun flow dépendant de données publiées.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowUpdateMetadataCommandHandler.kt:155](#)

Validation des ressources déclencheuses.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowUpdateMetadataCommandHandler.kt:202](#) Diff et événements de trigger.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/schedule/query/ListAvailableTriggersQueryHandler.kt:32](#) Découverte paginée et chemin racine.

[monorepo · services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/definition/event/DataProcessKafkaConsumer.kt:79](#) Consommation des schedules process-data.

Demandes de création de datasets et compilation des collectes

COMPORTEMENT OBSERVÉ

POST /datasets et /datasets/bulk préparent des demandes de collecte après décision IAM sur les cibles du payload. Le batch doit être non vide et viser un seul projet. Les fréquences PLANNED et les triggers événementiels sont refusés dans ce parcours. La validation des triggers porte sur tout le batch, mais les événements sont ensuite émis dataset par dataset et le controller documente l'absence d'atomicité globale. À réception d'une collecte CREATED ou UPDATED, le service attend brièvement le miroir du dataset, vérifie qu'il existe des entrées valides puis produit une CollectVersion. Il convertit CSV, XLSX, JSON, XML et Parquet ainsi que schéma cible, préparations et modes APPEND/OVERRIDE.

UTILISATEURS ET RÔLES

Utilisateur de collecte, data-collect et IAM.

INTERFACES DANS LE CODE

POST /datasets ; POST /datasets/bulk ; DatasetCollectEvent ;
DataflowCreateOrUpdateCollectDatasetCommandHandler.

DONNÉES ET ÉTATS

Commande de dataset créée par événement ; miroir DatasetBranch ; CollectVersion cachée avec executionProjectId réel, script et manifeste de sortie.

DÉPENDANCES ET ÉCHANGES

IAM dataset creation authorization, data-collect, data-model, stockage des fichiers importés, process-data et templates de collecte.

COMPORTEMENT CIBLE PROPOSÉ

Datasets et Connectors Rust prennent les commandes de collecte. Flows compose les transformations ; Writer écrit les candidats. Le résultat et le checkpoint source publié sont autoritaires dans l'édition Iceberg ; PostgreSQL garde la commande et les reçus.

MIGRATION DE CETTE CAPACITÉ

Sortir la génération de scripts Spark du parcours de collecte. Remplacer le batch partiel sans reçu par une commande avec résultat par item et clé d'idempotence. Reprendre chaque préparation et règle de schéma avant changement de moteur.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Interrompre un batch au troisième dataset puis le rejouer. Chaque item déjà accepté garde le même résultat, les autres reprennent et le client voit le détail. Une collecte sans entrée valide ne crée pas de publication vide implicite.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/dataset/controller/DatasetController.kt:31](#) Routes, décision IAM et limite de batch documentée.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/dataset/command/CreateDatasetWithDatasourceCommandHandler.kt:86](#) Validation globale puis émissions individuelles.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/command/DataflowCreateOrUpdateCollectDatasetCommandHandler.kt:28](#) Attente du miroir, entrées valides et CollectVersion.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/event/DataCollectKafkaConsumer.kt:71](#) Formats, types de source et préparations.

DD-20 **Cible non qualifiée**

Opérations techniques sur datasets et schémas

COMPORTEMENT OBSERVÉ

Les événements de data-model peuvent générer des versions techniques de suppression de table/branches, vidage et retouche de colonnes. La suppression supprime le miroir local puis produit une version DELETE par dataset avec branches et projet réel ; si un tombstone sans branches arrive, elle retire le miroir sans relancer un script. Le vidage exige un miroir correspondant à la branche. Une mise à jour compare les anciens/nouveaux schémas, conserve le schemaMode utile et peut générer suppression, renommage ou cast de colonnes. Une garde refuse une suppression massive qui ferait passer un schéma existant d'au moins deux colonnes à moins de deux. Ces opérations sont des exécutions, pas de simples modifications de métadonnées.

UTILISATEURS ET RÔLES

Data-model, administrateur de données et runtime technique.

INTERFACES DANS LE CODE

DeleteDatasetCommandHandler ; EmptyDatasetCommandHandler ; UpdateDatasetCommandHandler ; VersioningAction VERSION_DELETE, VERSION_EMPTY_DATASET, VERSION_UPDATE_COLUMN.

DONNÉES ET ÉTATS

Miroirs DatasetBranch et versions techniques cachées ; scripts et manifestes DELETE/REPLACE selon opération ; identifiant de projet réel séparé des références synthétiques.

DÉPENDANCES ET ÉCHANGES

Événements data-model, générateur de templates, S3 et process-data.

COMPORTEMENT CIBLE PROPOSÉ

Datasets Rust conserve les commandes métier ; Writer/Publisher applique WAP pour mutations de contenu et schéma. Maintenance traite expiration et GC avec plan audité et gardes de rétention. Une demande de suppression logique ne donne pas au worker le droit de drop de catalogue.

MIGRATION DE CETTE CAPACITÉ

Classier chaque action historique en suppression logique, nouvelle édition vide, évolution de schéma ou destruction physique. Importer les traces et remplacer les versions synthétiques par des RunSpec techniques explicitement typés.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un vidage validé publie une nouvelle édition vide et l'ancienne reste lisible selon rétention. Un cast non compatible ou une suppression de colonnes non approuvée laisse la dernière édition publiée intacte. Le GC refuse un fichier encore référencé par un lecteur épinglé.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/dataset/command/DeleteDatasetCommand.kt:29](#) Tombstone, nettoyage miroir et versions de suppression.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/dataset/command/EmptyDatasetCommandHandler.kt:22](#) Vidage par branche et projet réel.

[monorepo · services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/dataset/command/UpdateDatasetCommandHandler.kt:67](#) Diff de colonnes et garde de schéma.

DD-21 **Cible non qualifiée**

Retour d'exécution dans les graphes et historique de scripts

COMPORTEMENT OBSERVÉ

Data-dev consomme les événements de process-data pour conserver états d'exécution, schedules et clusters utilisés par l'éditeur. Le domaine expose les statuts CREATED, PENDING, RUNNING, FAILURE, SUCCESS, SKIPPED, WARNING et CANCELED. Les graphes peuvent mettre à jour les états des nœuds/sorties depuis ces événements. Les lectures de dernière exécution servent au résumé de publication et au dernier script exécuté d'une transformation. Ce miroir de développement n'est pas l'autorité d'exécution ni la preuve qu'un dataset a été publié après audit.

UTILISATEURS ET RÔLES

Éditeur, utilisateur consultant un flow et process-data.

INTERFACES DANS LE CODE

DataProcessKafkaConsumer ; UpdateGraphExecutionStateCommandHandler ; GET /dataflows/{id}/transformations/{transformationId}/lastExecutedScript ; GET /dataflows/{id}/publish/lazy.

DONNÉES ET ÉTATS

Execution miroir, états de graphe et références aux versions ; clusters/schedules miroir.

DÉPENDANCES ET ÉCHANGES

Protobuf d'exécutions process-data, stockage des versions et graphes.

COMPORTEMENT CIBLE PROPOSÉ

Flows Rust expose une projection reconstruisible depuis le journal de runs et les éditions approuvées. Frontends présente séparément état du calcul, état d'audit et état de publication.

MIGRATION DE CETTE CAPACITÉ

Conserver les identifiants de transformation, script, job et version dans les événements de remplacement. Importer les états sans les interpréter comme des preuves WAP. Prévoir la réparation de projection par replay.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Rejouer les événements de run dans un ordre retardé. Un état terminal ne redevient pas RUNNING et le graphe pointe vers le bon run. Un job réussi avec publication refusée apparaît comme calcul réussi et données non publiées.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/definition/event/DataProcessKafkaConsumer.kt:59](#) Consommateurs d'exécution, schedule et cluster.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/definition/aggregate/DataflowDefinition.kt:132](#) Vocabulaire des statuts.

[monorepo · services/data-dev/app/data-dev-](#)

[domain/src/main/kotlin/com/cleyrop/data/dev/domain/branch/command/UpdateGraphExecutionStateCommandHandler.kt:89](#) Projection des états dans le graphe.

[monorepo · services/data-dev/app/data-dev-](#)

[app/src/main/kotlin/com/cleyrop/data/dev/app/definition/controller/DataflowGetterController.kt:181](#) Lecture du dernier script exécuté.

Événements fiables, faits IAM et traitement des messages en échec

COMPORTEMENT OBSERVÉ

Les producteurs de versions et de faits d'autorisation utilisent une outbox PostgreSQL. Le relay réclame des lignes par verrou SKIP LOCKED, attend l'acquittement Kafka puis marque sentAt. Pour les messages avec clé, sa requête bloque les successeurs tant qu'un prédécesseur du même topic/clé n'est pas envoyé. Un envoi réussi suivi d'une panne avant commit peut être livré de nouveau. Les messages en échec sont parqués avec statut, cause et informations d'origine. Data-dev dispose d'un drainer, d'un probe de dépendances, de retrait après tombstone, d'expiration et d'un endpoint JMX de census/replay. Les mutations administratives ne sont pas exposées comme endpoints web génériques dans la configuration lue.

UTILISATEURS ET RÔLES

Consommateurs de domaine, IAM et opérateurs de reprise.

INTERFACES DANS LE CODE

OutboxPublisher/OutboxRelay ; DeadLetterEventConsumer/DeadLetterEventDrainer ; JMX dlt-replay ; AuthzFactOutboxPublisher.

DONNÉES ET ÉTATS

outbox_event ordonné par relay_sequence ; dead_letter_event avec reçu de traitement et statut ; faits de ressources pour IAM.

DÉPENDANCES ET ÉCHANGES

Kafka/Protobuf et JSON de certains événements, PostgreSQL, IAM comme writer des tuples.

COMPORTEMENT CIBLE PROPOSÉ

Audit et Flows Rust conservent outbox/inbox et schémas d'événements versionnés. Le transport final n'a aucune JVM ; les clés de déduplication et d'ordre sont contractuelles. Les reçus de publication ou d'effets restent durables sans dépendre d'un fan-out UI.

MIGRATION DE CETTE CAPACITÉ

Importer outbox non envoyée et messages parqués sans modifier leurs payloads. Construire des adaptateurs de double lecture temporaire avec un seul writer par ressource. Qualifier les reprises et l'ordre par agrégat avant remplacement du transport Kafka.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Faire tomber le relay après acquittement distant et avant sentAt. Le replay est déduplicqué par le consommateur et ne double ni version ni autorité. Un message dépendant d'un projet absent reste parqué puis reprend quand le projet apparaît.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/common/outbox/OutboxEventJPA.kt:11](#) Ordre par clé et verrou SKIP LOCKED.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/common/outbox/OutboxRelay.kt:25](#) Envoi, acquittement, erreurs et rétention.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/common/deadletter/DeadLetterEventDrainer.kt:143](#) Drainer et sweep des messages parqués.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/common/deadletter/DeadLetterReplayEndpoint.kt:37](#) Administration JMX de census et replay.

Migrations de code, réconciliation et exploitation

COMPORTEMENT OBSERVÉ

Le service combine migrations Flyway et migrations de code suivies par une table cleyrop_migration. Il possède des modes de démarrage dédiés migrate et reconcile, dont un parcours de réparation des applications et de leurs membres. Un reconciler de comptes de service reprend les projets qui ne sont pas encore provisionnés dans IAM. Les déploiements incluent des sondes, un port de management, un espace temporaire pour les archives, un PVC optionnel et des règles réseau de gateway. Les métriques couvrent notamment outbox, DLT et échecs de verrouillage Git. L'état opérationnel dépend donc de PostgreSQL, des artefacts S3 et d'effets distants, pas du seul pod API.

UTILISATEURS ET RÔLES

Opérateur de plateforme et automatisation de migration.

INTERFACES DANS LE CODE

MigrateMode ; ReconcileMode ; UpgradeReconcile ; CleyropMigrationRunner ; ServiceAccountReconcileJob ; health/prometheus et endpoints JMX configurés.

DONNÉES ET ÉTATS

Journal cleyrop_migration, marqueurs de provisioning du projet, miroir d'applications, outbox/DLT, archives en S3.

DÉPENDANCES ET ÉCHANGES

Flyway, PostgreSQL, IAM, Git, applications, Kafka et S3.

COMPORTEMENT CIBLE PROPOSÉ

Migration Rust orchestre des étapes idempotentes avec préconditions, journal et validation. Audit conserve les preuves de conversion. Maintenance suit rétention et sauvegarde des commandes, graphes et artefacts.

MIGRATION DE CETTE CAPACITÉ

Exporter un inventaire vérifiable des tables et artefacts, prendre un point de reprise cohérent puis importer par cohortes. Le retrait de JVM inclut Flyway embarqué et les migrations Kotlin restantes. Garder les outils legacy isolés uniquement pendant la transition.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Interrompre l'import entre deux lots puis reprendre. Les IDs et graphes importés restent identiques ; aucun effet Git/IAM n'est doublé. Une restauration de sauvegarde retrouve les artefacts référencés et les commandes non terminées.

Références de l'analyse

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/runmode/UpgradeReconcile.kt:55](#)
Actions de backfill et applications.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/runmode/ReconcileMode.kt:51](#)
Mode de réconciliation dédié.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/project/service/iam/ServiceAccountReconcileJob.kt:96](#) Réconciliation du provisioning IAM.

[monorepo · services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/codemigration/runner/CleyropMigrationRunner.kt:30](#) Migrations de code au démarrage.

[monorepo · services/data-dev/chart/templates/deployment.yaml:68](#) Sondes et montage des volumes.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Flows Rust possède drafts, révisions exécutables, branches, graphes, MR, résolution, activation et manifestes. La révision inclut digest du code/modules et version du compilateur.
- IAM Rust possède projets, membres et droits. Les contrats conservent une vérification de cohérence entre l'objet nommé par une route, le projet du payload et les objets réellement persistés.
- Définir un RunSpec qui nomme principal, projet réel, version de flow, éditions d'entrée, profils de ressources, sorties autorisées, secret refs et deadline. Les huit kinds sont TABLE, INCREMENTAL, VIEW, FILESET, MODEL, EMBEDDINGS, FEATURESET et STREAM.
- Writer médie le staging et Publisher médie la publication. Flows n'accorde pas au worker un accès REST générique table.modify ; le droit inclut commit/drop et dépasserait le besoin du job.
- La définition publiée et la donnée publiée ont des cycles distincts. L'historique de publications approuvées ne se reconstruit pas en listant tous les snapshots.
- Les événements de dépendance et les checkpoints publiés référencent l'édition Iceberg autoritaire. PostgreSQL reste autoritaire pour drafts, commandes, identités et reçus d'effets.
- Applications traite provisionnement Git/notebooks comme effets idempotents, avec état de réparation. Notifications traite le fan-out UI séparément des preuves durables.

Plan de données proposé

- Compiler le low-code vers un plan typé sans Spark. Chaque opérateur doit avoir une sémantique prouvée sur schéma, nulls, types, dates, ordre et erreurs.
- Exécuter SQL et transformations tabulaires dans le moteur Rust retenu après campagnes. Python/SQL utilisateur reste un format d'usage, dans des processus bornés et sans API Spark conservée par défaut.
- Stager des fichiers immuables, produire un candidat avec origin.* au summary puis faire auditer ce candidat. La preuve post-candidat est liée durablement, sans réécriture du summary par fast-forward.
- Pour un flow à plusieurs sorties, produire une release qui est elle-même une édition Iceberg avec checkpoint source/epoch/range/participants et lecteurs épinglés.
- Un preview est un résultat de run temporaire avec portée et rétention, jamais une exception aux règles d'accès. Les résultats STREAM viennent d'un journal durable et franchissent WAP.
- EXPORT est un effet avec reçu et EXPOSURE un consommateur déclaré. Ni l'un ni l'autre n'ajoute un neuvième kind de sortie.

Découpage du core Rust

- Proposition flows-model : contrats serde pour graphe, étapes, branches, révisions, modules, manifestes et types d'erreurs, avec import du JSON historique.
- Proposition flows-compiler : tri du graphe, validation des opérateurs, calcul des schémas, dépendances d'artefacts et génération de plans SQL/Arrow ou invocation Python.
- Proposition flows-revisions : sauvegarde à révision attendue, diff sémantique, MR, règles de résolution et activation de définition.
- Proposition flow-run-contract : RunSpec immuable, liaisons vers Writer/Publisher, preuves origin.*, entrées épinglées et sorties déclarées.

- Proposition applications-adapters : Gitea, notebooks, comptes de service et procédures d'offboarding sous commande durable.
- Proposition event-runtime : outbox/inbox, déduplication, ordre par agrégat, DLT et réconciliation indépendants du transport final.
- Réutilisation d'un moteur Rust existant à qualifier, sans annoncer qu'Icegres ou Eidos couvrent les opérateurs de ce service. Leur capacité concrète est hors de cette lecture.

Identité, autorisation et isolation

- Garder séparés appelant, auteur de version, propriétaire et principal de run. Les transferts ne réécrivent pas les preuves historiques.
- Conserver le refus des payloads croisant projet et ressource. La canonicalisation legacy/IAM est obligatoire avant comparaison.
- La directive générale d'autorisation gateway ne suffit pas à décrire le code : suppression de branche, création de MR, éligibilité du propriétaire et création de dataset gardent des contrôles locaux. Ils doivent entrer dans la matrice de migration IAM.
- Les secrets EnvVar sont masqués dans le DTO mais visibles au DAO et incorporés aux scripts générés. La migration doit retirer ces valeurs des artefacts et adopter des références de secrets versionnées.
- Les clés SSH ont un convertisseur de chiffrement ; leur migration et rotation doivent préserver la compatibilité avec la clé partagée existante sans export de clé dans le dossier.
- La portée preview n'est fournie qu'aux clients internes configurés et exige un manifeste complet. Les travaux de migration doivent réparer tout écart entre génération et persistance de cette preuve.
- Un worker cible ne peut ni publier ni drop une table. Les vérifications UUID/ref/schema/spec/ascendance appartiennent au Publisher et ne sont pas présentées comme un fencing Hemera natif.

Exploitation et reprise

- Deux modules Gradle séparent domaine et adaptateurs Spring. Les versions lues sont celles du dépôt ; aucune version déployée n'a été interrogée.
- La sauvegarde/restauration doit couvrir PostgreSQL et artefacts S3, ainsi que le rapprochement avec les dépôts Git, clés chiffrées et applications externes.
- Les sondes et le management Prometheus sont définis dans le chart et les propriétés. L'existence d'un PodMonitor dépend des conditions de chart et de la CRD de monitoring.
- Le chart permet un stockage persistant optionnel ; sa documentation indique que cette option impose un seul replica et une stratégie Recreate. L'espace temporaire sert notamment aux ZIP générés.
- L'outbox réclame les événements par verrous et conserve l'ordre par clé, mais les consommateurs doivent supporter la relivraison. Les métriques DLT et de provisioning sont nécessaires au pilotage de bascule.
- Les migrations Kotlin/Flyway et les modes migrate/reconcile constituent une dette JVM opérationnelle à remplacer. Les migrations de code doivent être qualifiées en reprise partielle.
- Un échec de verrouillage Git peut laisser un compte distant ouvert malgré le miroir révoqué. Le dossier propose un suivi durable de cet effet et une réconciliation avant fermeture d'offboarding.
- Aucun test Gradle ou Helm n'a été exécuté pendant cet audit en lecture seule.

Séquence de migration

- 1. Inventorier IDs legacy/IAM, graphes, branches actives/supprimées, versions, MR, modules, variables, artefacts S3 et événements non terminés.
- 2. Construire un importeur sans secrets en clair qui conserve IDs, dates, auteurs et liens d'artefacts. Réconcilier les versions dépourvues de manifeste avec leur preuve historique.
- 3. Formaliser les contrats API et de payload observés, dont 410 projet, erreurs de sauvegarde concurrente, batch partiel, branches restaurables et MR dépendantes.
- 4. Porter le modèle et le compilateur de low-code vers Rust, puis qualifier chaque opérateur et chaque format. Classer les programmes PySpark qui exigent une réécriture utilisateur.
- 5. Introduire RunSpec et le protocole Writer/Audit/Publisher WAP. Séparer activation d'une définition et publication de résultats, avec sorties multiples et checkpoints publiés dans Iceberg.
- 6. Migrer secrets/modules en artefacts immuables, puis déplacer Git/notebooks et leurs effets vers Applications. Finir le transfert des projets et membres vers IAM.
- 7. Basculer par projet avec un seul writer de drafts, compare de projections et replay d'outbox. Maintenir des lecteurs temporaires de l'historique sans autoriser les anciens runtimes à republier.
- 8. Retirer générateur Spark, clients JVM, Spring/Flyway Kotlin et transport Kafka JVM seulement après les campagnes de résultats, révocation, reprise et restauration.

Écarts et décisions encore ouverts

- Preview : le handler lit VersionRepository mais ne sauvegarde pas la version produite ; le producteur ne fait qu'émettre. PreviewScopeQueryHandler exige pourtant findById. Vérifier l'existence d'un autre chemin de persistance et qualifier ce raccord avant d'en faire un contrat cible.
- Publication : le code lit la dernière exécution pour le résumé sans exiger son succès dans le handler de publication. La politique d'activation d'un flow doit être décidée séparément de WAP.
- Variables secrètes : le périmètre historique d'artefacts qui incorporent des valeurs n'a pas été inspecté, conformément à la consigne. Une campagne sécurisée d'inventaire est requise.
- Concurrence : baseVersionId reste optionnel et certains chemins actualisent une GraphVersion. Il faut décider la durée de compatibilité de ces écritures historiques avant immutabilité stricte.
- Autorisation : le code conserve des règles de propriétaire et d'éligibilité dans des handlers malgré la note générale gateway. Réconcilier code, routes IAM et contrat métier avant suppression de ces règles.
- Suppression de dataset : distinguer les événements sans branches, qui ne génèrent pas de script, et les suppressions de contenu. Aucun tombstone importé ne doit être converti en destruction physique automatique.
- La compatibilité des fonctions Python/PySpark privées, modules utilisateurs et images installées dépend de données client non lues. Aucun taux de compatibilité n'est annoncé.
- Le couplage observé à Nessie et aux noms Spark de branches doit être traduit ; un renommage de ref seul ne fournit pas un protocole de publication autorisé et audité.

Références de l'analyse

[monorepo](#) · [services/data-dev/app/gradle/libs.versions.toml:23](#) Versions Kotlin et Spring dans le dépôt.

[monorepo](#) · [services/data-dev/app/data-dev-app/build.gradle.kts:24](#) Toolchain JVM 21.

[monorepo](#) · [services/data-dev/app/data-dev-domain/src/main/kotlin/com/cleyrop/data/dev/domain/version/aggregate/Version.kt:19](#) Familles de versions.

[monorepo](#) · [services/data-dev/app/data-dev-app/src/main/kotlin/com/cleyrop/data/dev/app/definition/service/ScriptGeneratorServiceImpl.kt:339](#) Compilation d'une version exécutable.

[monorepo](#) · [services/data-dev/chart/templates/deployment.yaml:50](#) Processus non root, sondes et volumes.

[monorepo](#) · [services/data-dev/chart/values.yaml:239](#) Contraintes du mode persistence.